

Comprehensive HTML Feature Showcase

This document demonstrates every HTML element supported by the AIPDF HTML converter. It tests the full rendering pipeline from HTML input through semantic XML to production-quality PDF output, covering headings, paragraphs, lists, tables, code blocks, blockquotes, and figures.

Paragraphs and Text

The AIPDF format preserves document structure semantically. Paragraphs are the primary text block, capturing prose content. Long paragraphs are word-wrapped using precise Helvetica character-width metrics to ensure accurate line breaks in the rendered PDF output.

Multiple consecutive paragraphs are each emitted as individual semantic paragraph blocks. Each block receives a unique stable ID for cross-referencing and ordered extraction. The semantic layer stores no embeddings, model output, or executable content.

AIPDF files are valid PDF 1.7 documents. Legacy PDF readers display the visual content stream while ignoring the embedded associated file. AI-aware parsers detect the marker, extract the Brotli-compressed payload, and parse the structured XML directly.

Unordered Lists

Key features of the AIPDF format:

- Brotli-compressed semantic XML embedded as an associated file
- Full PDF 1.7 compliance — readable by any PDF viewer

- Zero OCR required for machine-readable consumption
- Schema-validated structure with stable block IDs
- Three SDK surfaces: Rust (core + CLI), Python, and TypeScript
- Export to XML, Markdown, Markdown AST JSON, and ONTO columnar format

Ordered Lists

Steps to produce an AIPDF document:

1. Author content in XML, Markdown, HTML, or Typst format
2. Run the CLI: `aipdf build input.html --render full -o output.aipdf`
3. Distribute the .aipdf file — legacy readers see the PDF, AI readers extract the XML
4. Use the CLI, Python SDK, or TypeScript SDK to query the semantic layer
5. Export to your preferred format for downstream processing

Tables

Compression Method Comparison

METHOD	RATIO	SPEED	BEST FOR
Brotli 6	85%	Fast	General documents
Brotli 11	92%	Slow	Archive and storage
Gzip 9	78%	Medium	Legacy compatibility
Zstd 3	83%	Very fast	Real-time encoding

SDK Support Matrix

Feature Support Across SDK Surfaces

FEATURE	RUST CLI	PYTHON SDK	TYPESCRIPT SDK
Build from XML	Yes	Yes	Yes
Build from HTML	Yes	Yes	Yes
Build from Markdown	Yes	Yes	Yes
Build from Typst	Yes	Yes	Yes
Export to Markdown	Yes	Yes	Yes
Export to ONTO	Yes	Yes	Yes
Full render mode	Yes	No	No

Code Blocks

Rust Example

```
use aipdf::{build_aipdf, BuildOptions, RenderMode, PageOptions};

fn main() -> aipdf::Result<()> {
    let xml = std::fs::read_to_string("document.xml")?;
    let pdf = build_aipdf(&xml, &BuildOptions {
        title: "My Document".to_string(),
        render: RenderMode::Full,
        page: PageOptions::a4(),
        ..Default::default()
    })?;
    std::fs::write("document.aipdf", pdf)?;
    Ok(())
}
```

Python Example

```

import aipdf

# Open an existing document
doc = aipdf.open("document.aipdf")

# Export in multiple formats
print(doc.to_markdown())
print(doc.to_onto())

# Access structured blocks in reading order
for block in doc.get_reading_order():
    print(f"{block['kind']}: {block['text'][:60]}")

# Build from HTML source
from aipdf import build_from_source
xml = build_from_source(html_content, "html")
pdf_bytes = aipdf.build(xml, title="My Doc")

```

TypeScript Example

```

import { open, buildFromSource } from 'aipdf';

// Build from HTML source
const xml = buildFromSource(htmlContent, 'html');

// Open and query
const doc = await open('document.aipdf');
const blocks = doc.getReadingOrder();
const markdown = doc.toMarkdown();
const onto = doc.toOnto();

console.log(`Loaded ${blocks.length} semantic blocks`);

```

Shell Commands

```
# Build with full rendering from HTML
aipdf build samples/comprehensive.html --render full --page-size letter -o out

# Inspect the embedded semantic layer
aipdf inspect out.aipdf

# Validate structural correctness
aipdf validate out.aipdf

# Export to ONTO format for LLM ingestion
aipdf export out.aipdf --format onto
```

Blockquotes

The PDF format has become the de facto standard for document exchange. AIPDF extends this foundation by embedding a machine-readable semantic layer, enabling AI systems to access structured content without OCR or heuristic parsing.

Brotli achieves significantly better compression ratios than gzip for text content at comparable decompression speeds, making it ideal for the XML semantic payloads embedded in AIPDF files.

AI-native document formats must balance human readability, machine accessibility, and backward compatibility. AIPDF demonstrates that all three are achievable in a single file without compromise.

Figures and Images



MAPPA Crunchyroll

LE 22 OCTOBRE EXCLUSIVEMENT AU CINÉMA

SONY



Figure 1: The AIPDF processing pipeline. Source documents in XML, Markdown, HTML, or Typst are converted to semantic XML, Brotli-compressed, and embedded in a standards-compliant PDF 1.7 file. Legacy PDF readers display the visual content; AI-aware parsers extract the structured semantic layer directly.



MAPPA Crunchyroll

LE 22 OCTOBRE EXCLUSIVEMENT AU CINÉMA

SONY



Figure 2: Compression ratio versus encode time for Brotli at levels 1–11. Level 6 offers the best tradeoff for document workflows: near-maximum compression at near-minimum latency.

Nested Sections

Subsection: Security Model

The AIPDF security model is based on three principles that prevent payload injection attacks:

- Sanitization rejects DOCTYPE declarations, inline script content, and prompt injection markers
- Structural validation enforces schema compliance before embedding
- The semantic layer never contains embeddings, model output, or executable content

Disallowed Markers

The sanitizer rejects any XML payload containing the following strings regardless of case:

- DOCTYPE
- xml-stylesheet processing instruction
- script elements
- PDF JavaScript and Launch action types
- LLM injection directives (prompt prefix strings)
- AI system override strings (instruction injection patterns)

IMPLEMENTATION DETAIL

Character widths for word-wrap are measured in 1/1000 pt units using the complete Adobe Helvetica metrics table for all ASCII printable characters. Both regular and bold weights are

supported with separate width tables.

PDF OBJECT STRUCTURE

The minimal render mode produces exactly 9 PDF objects. The full render mode produces a dynamic number of objects: catalog, page tree, one page object per page, font resources, content streams, XMP metadata, embedded file spec, and the Brotli-compressed semantic stream.

Container Elements

ONTO Export Format

The ONTO format is a columnar, token-efficient representation designed for LLM ingestion. It uses pipe and caret delimiters for table data and applies scalar encoding rules to prevent delimiter collision in cell content.

```
Document title Comprehensive AIPDF Feature Showcase
Sections id level title
s1 1 Introduction
s2 2 Architecture
Blocks id kind page text
b1 paragraph 1 The AIPDF format...
Tables id caption rows
t1 Compression Brotli^85%^Fast|Gzip^78%^Medium
```

The ONTO format reduces token count by 60–80% compared to raw XML, making it the preferred export format for LLM-based document analysis pipelines where context window efficiency matters.

Implementation Notes

The HTML converter uses the scraper crate (html5ever-based) for spec-compliant DOM parsing. This handles malformed HTML, implicit element closure, and Unicode normalization correctly — all critical for real-world document conversion where authors do not always produce well-formed markup.

Performance Benchmarks

Measured on Apple M2 Pro, single-threaded, release build:

Build Time vs. File Size (Minimal Render Mode)

INPUT XML SIZE	BUILD TIME	AIPDF SIZE	OVERHEAD
1 KB	2 ms	1,240 bytes	24%
10 KB	4 ms	7,800 bytes	8%
100 KB	18 ms	68,000 bytes	5%
1 MB	142 ms	620,000 bytes	3.8%

Conclusion

The AIPDF format achieves its core design goal: a single file that serves both human readers via standard PDF rendering and AI systems via embedded structured XML. The semantic layer adds minimal overhead while enabling precise, OCR-free content extraction across all supported output formats.

The production-quality PDF renderer produces multi-page documents with styled headings, word-wrapped paragraphs, grid-based tables, syntax-highlighted code blocks, bulleted and

numbered lists, citation sidebars, and footnote annotations — all using only base-14 PDF fonts with no external dependencies.